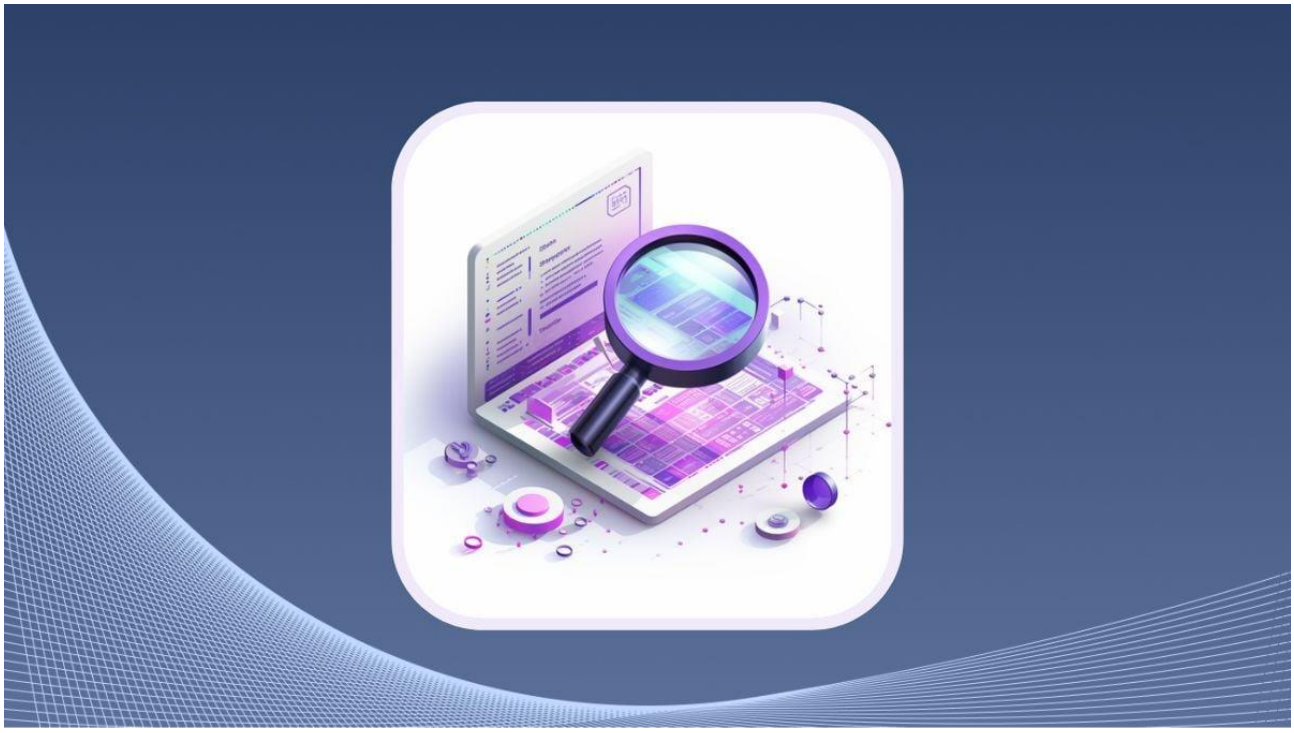


Clean Code Best Practices

Version 1.0 | 18th July 2024

AZILEN TECHNOLOGIES PVT. LTD.
UNITED STATES | GERMANY | SWITZERLAND | INDIA

Copyright © AZILEN TECHNOLOGIES PVT. LTD.

PRIVATE & CONFIDENTIAL

Version History **4**
1 Introduction **5**
2 What is Clean Code? **5**
2.1 Why is it important to follow Clean Code Practices? 5
2.2 A Good Developer is like an Author 5
3 Three Principles of Clean Code **6**
3.1 Right Tool for the Job 6
3.2 High Signal to Noise ratio 6
3.3 Self-Documenting Code 6
4 Best Practices for Folder/Package Structure **7**
4.1 Folders/Packages by Type 7
4.2 Folders/Packages by Feature 7
5 Best Practices on Naming Convention **9**
5.1 Naming Classes 10
5.2 Naming Methods 10
5.3 Naming Variables – Booleans 11
5.4 Naming Variables – Maintain Symmetry 11
6 Best Practices on Conditions **12**
6.1 Boolean Comparison 12
6.2 Assign Booleans Implicitly 13
6.3 Ternary is elegant 14
6.4 Magic Numbers 15
6.5 Complex Conditions 15
6.5.1 Intermediate Variable 15
6.5.2 Encapsulate Complex Conditions 16
6.6 Polymorphism over Enums 17
6.7 Be Declarative than iterative 18
6.8 Table Driven Methods 19
7 Best Practices on Functions / Methods **21**
7.1 Duplication 21
7.2 Excessive Indentation 22
7.2.1 Extract Method 23
7.2.2 Return Early 23
7.2.3 Fail Fast 24
7.3 Convey Indent 25
7.4 Do One Thing 25
7.5 How many parameters function should have? 26
8 Best Practices on Classes **27**
8.1 When to create classes? 28
8.2 High Cohesion 28

8.3 Detecting low cohesive classes 28
8.4 Primitive obsessions 29
8.5 Principle of Proximity 29
9 Best Practices on Comments 30
9.1 Bad Comments 30
9.1.1 Redundant Comments 30
9.1.2 Intent Comments 31
9.1.3 Apology Comments 31
9.1.4 Warning Comments 31
9.1.5 Zombie Code Comments 32
9.1.6 Divider Comments 32
9.1.7 Brace Tracker Comments 33
9.1.8 Defect Log Comments 33
9.2 What are Good Comments? 33
10 Best Practices on Logs 34
11 Staying Clean 34
11.1 When to refactor to Clean Code? 34
11.2 Code Reviews 34
11.3 Pair Programming 34
11.4 Volunteer Rule 34
12 Checklist 35

Version History

Version	Date	Amendment
1.0	2024-01-01	Ready for publication
2.0	2024-07-30	Added Branding, Checklist Section, layout changes

1 Introduction

*"**Truth can only be found in one place: the code** - Robert C. Martin"*

At Azilen, we are committed to write beautiful code for all the projects and clean code practice helps us to achieve this commitment.

In this document, clean code pointers are explained which are followed at Azilen.

At the end of the document, a checklist is introduced which can help an engineer to bring clean code practice in his routine.

2 What is Clean Code?

Clean Code is a reader-focused programming style that produces software that's easy to write, read and maintain.

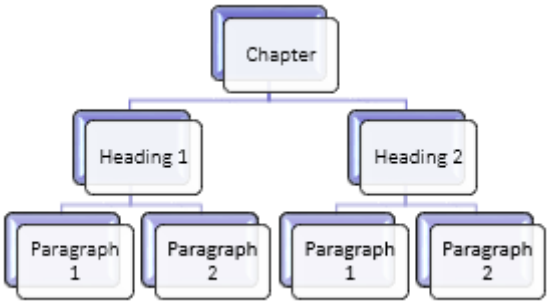
2.1 Why is it important to follow Clean Code Practices?

Any developer can write the code that a computer can understand. Good programmers write code that humans can understand.

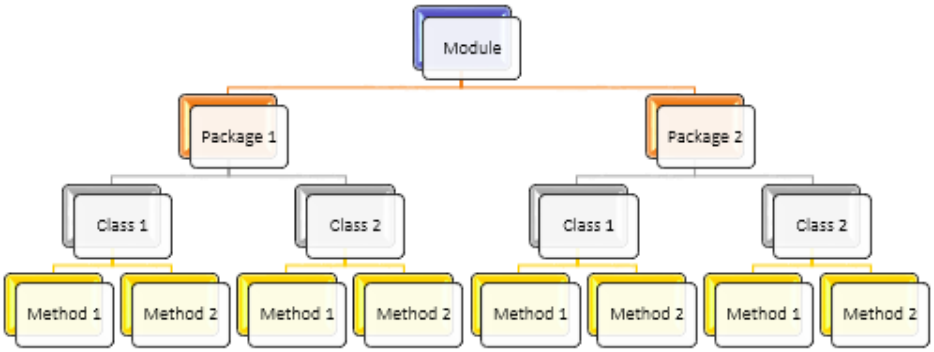
- Writing code is easy. But reading code is hard!
- Code is like writing a stand-up comedy script. If you have to explain it, it's not good.
- Dirty code creates Technical Debt and Technical Debt can kill your product.
- Short cuts can be harmful in long term.
- It's always a race between Deadline vs. Quality. Clean code provides a way to write code smartly with quality and achieve your deadlines.
- You don't want to be the bad example in your company by writing dirty code.
- It should be possible to give walkthrough of the code to the domain experts to get the meaningful inputs.
    - Although not all projects require code walkthrough with the domain experts, but if the code reflects the domain space and functional understanding of the team, it is possible to maintain the code for the longer duration.
- For a new developer, it should be easier to understand the feature and work on it asap. The effort for the code navigation should be minimal.
    - e.g. If new developer is asked to add some enhancement in payroll module, it should be easier for him to go through the code of the payroll module and understand it.

2.2 A Good Developer is like an Author

- An author writes book/article/text professionally.
- Each line is proofread multiple time.
- Author writes in a structured manner.
- Programmers are no different.



Author's Book Structure



Programmer's Code Structure

You as a programmer needs to be as professional as the authors when it comes to writing the code.

3 Three Principles of Clean Code

3.1 Right Tool for the Job

- You should not use Node.js for OOP based projects, Java is good for that.
- You should not use Java for AI/ML based projects, Python is good for that.
- You should not use MongoDB for relational data in a system that involves heavy financial transactions. PostgreSQL, MySQL or any such SQL based database is good for that.
- Writing SQL Queries in HTML and writing JavaScript is HTML is not a good idea. Manage your boundaries.
- Always strive for using right tool for the job.

3.2 High Signal to Noise ratio

- A lot of commented code, complex code, unreadable longer code etc are the Noise in the code.
- Readable code, simple code, small code, code with good variables and function names etc are the Signal in the code.
- Always strive for having High Signal to Noise Ratio in your code.
- Follow TED (**T**erse (Short), **E**xpressive, **D**o one thing) Rule for having High Signal to Noise Ratio in your code.

3.3 Self-Documenting Code

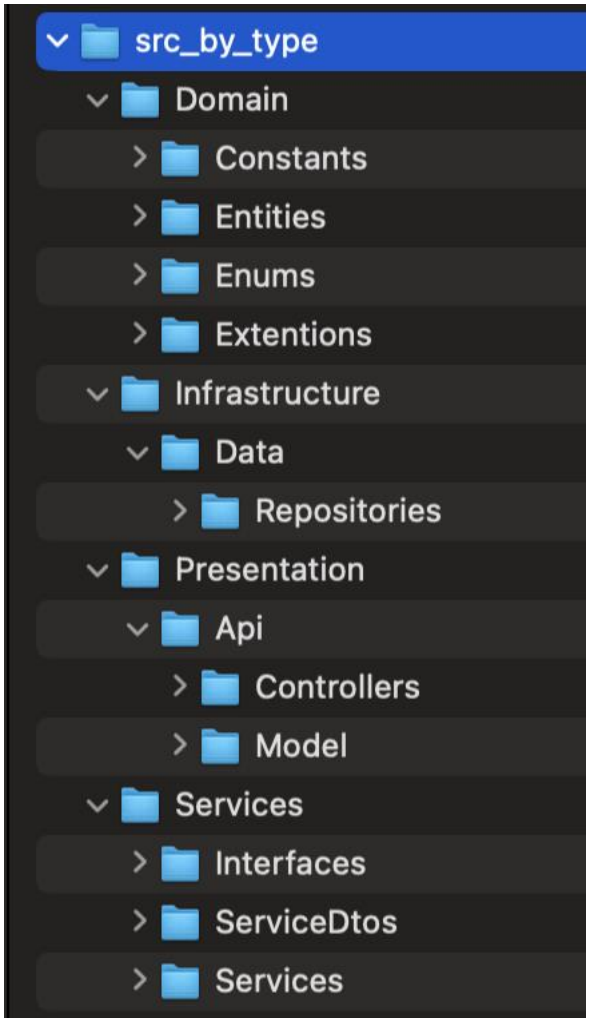
- Write code with Clear Intent, so that you don't have to explain it with comments.
- Use good Layers of Abstraction to represent the objects in your code.
- Use good level of Formatting on your code for Readability.
- Always Favor Code over Comments. It means, it should be possible for readers to understand your code without the need for comments. You can add comments if it can increase the value for the readers. E.g. You can add details about business scenarios which explain how the code would behave in a business environment. If you need to write some special logic because of a certain aspect of the external system, you can describe the reason for writing such logic via comment.

4 Best Practices for Folder/Package Structure

There are two ways to arrange your source code in the folders.

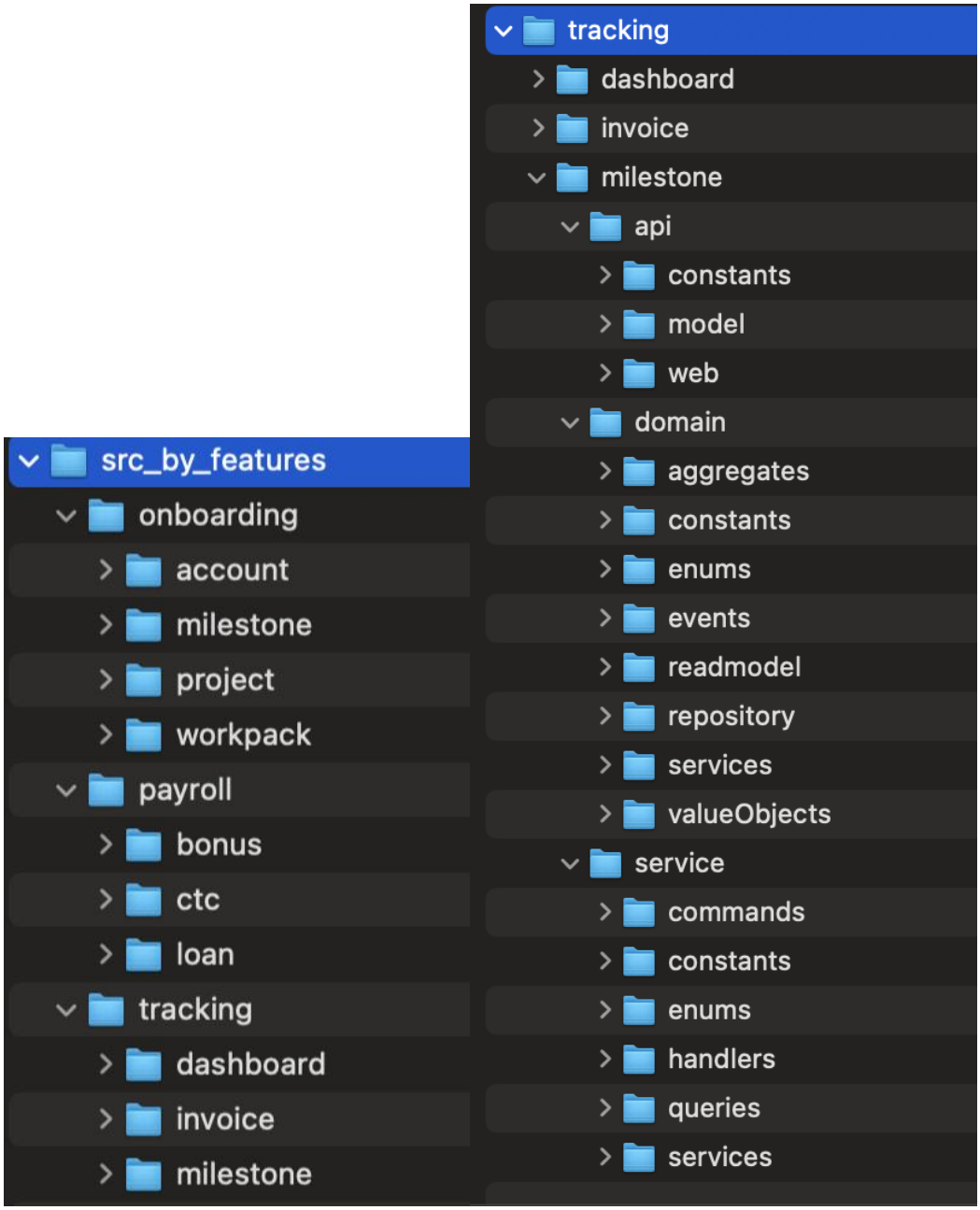
4.1 Folders/Packages by Type

- Folders are arranged from technical perspective e.g. controllers, services, models, entities and so on.
- Inside the type-based folders, if required feature wise sub folders are created. (depending on the complexity of the domain).
- The challenge with this structure is, implementation of any feature is spread across multiple places and can be difficult to understand the code.



4.2 Folders/Packages by Feature

- In this case, folders are created based on the functional features e.g. payroll, employees, courses etc.
- Within feature folder, subfolders are considered based on the type.
- If feature is complex in nature and has sub features, folders are also update based on that.
- Within one folder/package, if multiple types of files are available, group them appropriately. E.g. in Employee module, within model folder, you may have subfolders such as work experience, personal, professional etc. And in those subfolders, respective files can reside.



Unless any constraint from the language/technology side, “Folder/Package by Feature” is always the preferable option.

- It will make code with "Low coupling, high cohesion" which is desirable situation.
- The code will become easier to understand and adding new features is easier with this.
- Note: Within different modules, please avoid cyclic dependency, if possible. E.g. if Module A refers code of Module B, the code of Module B should not refer the code of Module A.

Handling of the common modules

- Instead of keeping “commonXXXX” class (e.g. commonConstants or commonService), it would be better to create those classes based on the functionalities it will contain.
- We should be extra careful while adding something in common space.

5 Best Practices on Naming Convention

Let's say you are reading a Book that has following sentence, will you be able to read what book is trying to say?

"**P** was very angry with **G** for insulting her **M**. **G** kicked **P**. He fell on the **C**."

No... Right??

Why..? Because you cannot relate to the story in the sentence as you cannot relate to characters or things like **P**, **G**, **M**, **C** etc.

And that is because these characters/things are not named properly.

Same is true with programming when you use bad variable/class/method names. Let's look at an example,

DIRTY

```
List<Double> p = Arrays.asList(24d,73.33d,60d,66.33d);
Double t = 0d;
for (Double i: p) {
    t+=i;
}

return t;
```

Can you figure out what above code is trying to achieve except it has some numbers and we are making total of it? Can you find what this program is about functionally?

No... Right??

Now let's look at this example,

CLEAN

```
List<Double> prices = Arrays.asList(24d,73.33d,60d,66.33d);
Double total = 0d;
for (Double price: prices) {
    total+=price;
}

return total;
```

Now we can see that program returns the **total of the given prices**. Good variable names tell the right story to the fellow developers who are reading your code.

5.1 Naming Classes

There are certain guidelines are there to name the classes.

- The class name should be a **Noun.**
  - It should be a thing that can be instantiated. E.g., User, Account, Product etc.
- Be Specific when naming the class.
  - Smaller
  - Cohesive Classes
- Single Responsibility
  - It should do one thing
- Avoid Generic Suffix
  - ProductInfo
  - AccountInfo

5.2 Naming Methods

Method names should be in a verb as it performs the action in the code, so by naming the method, the name itself should speak what action it is doing.

For an example: getRegisteredUsers, isValidDate, importTemplate, createUser.

Methods name should avoid using And, Or, If, as it gives bad impact of the code which is been implemented.

Totally avoid using of:

- AND
- OR
- IF

Example:

- SaveAndEmail
- LoadAndPrint
- ProcessorDeny
- SaveEmailIfUserFound

While naming the methods we should also think about the side effects from naming the methods.

- GetUser: should simply get the details of the user and not update it.
- IsValidUser: It should only verify the user and not save it directly.

Abbreviations

Abbreviations are short form of word, in earlier days space was costly so the naming methods were short, but today there is no issue of space, so we should take the name in a such a way that it should be meaningful, like registerUser() and not naming regisUsr() as it doesn't symbolise what regisUsr is.

Let's have look in another example.

Here we taken the variables in the form of 'i', 'j', 'p' and 'q', which doesn't give any information what they are about, instead of that we can define what they are and then use it.

```
for (int i = 0; i < p.length; i++) {
    for (int j = 0; j < q.length; j++) {
        System.out.println(p[i][j]);
    }
}
```

5.3 Naming Variables – Booleans

Naming the variables is one of the major criteria, as it leaves the impact when we trying to solve the problems. So, Booleans are the best datatypes which has two possible values either true or false. While naming the Boolean variables we just need to keep two things in mind while naming.

- Avoid choosing the names containing negations.
- Choose names like
  - isOpen
  - isStarted
  - isActive
  - loggenIn
- Avoid using these names
  - open
  - start
  - status
  - login

5.4 Naming Variables – Maintain Symmetry

When dealing with state variables that toggles, be symmetrical.

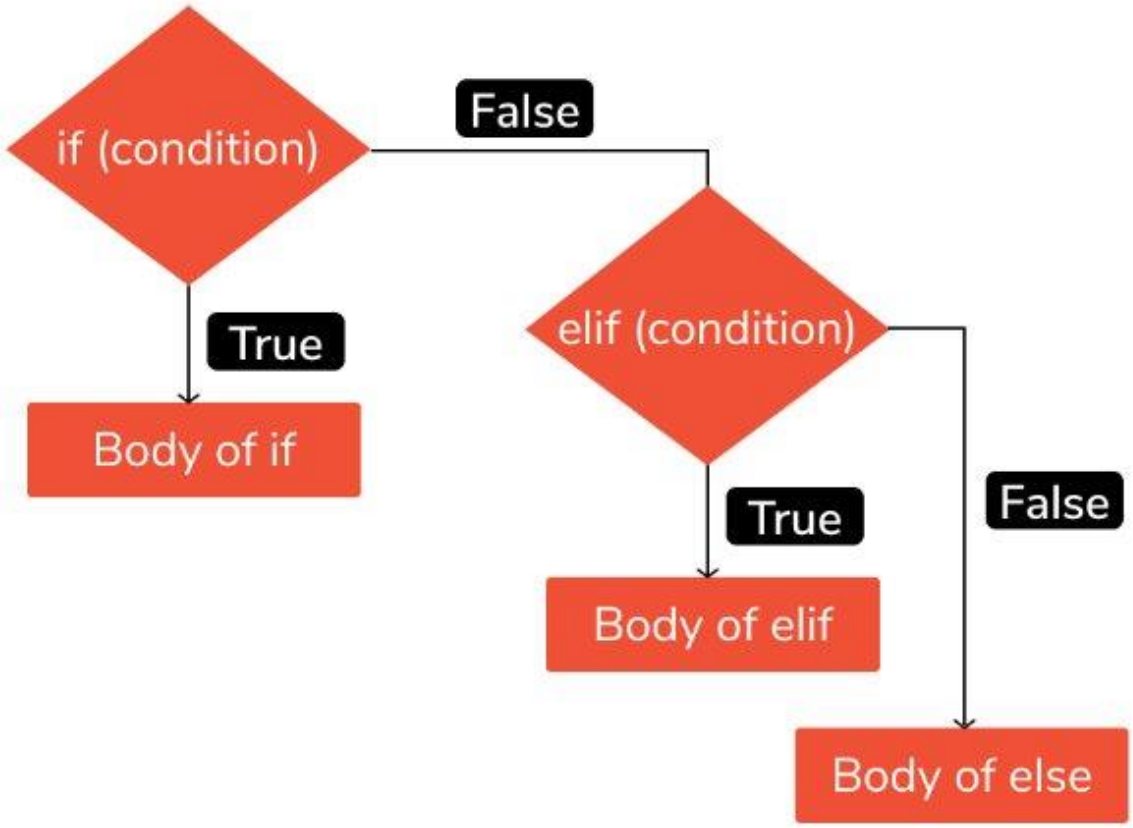
Clean

- on <-> off
- fast<->slow
- lock<->unlock
- min<->max

Dirty

- on<->disable
- quick<->slow
- lock<->open
- slow<->max

6 Best Practices on Conditions



6.1 Boolean Comparison

By default, If conditions are in their true state, so there is no point in double checking the condition.

Dirty

```
if(loggedIn == true) {
    //Do something
}
```

Clean

```
if(loggedIn) {
    //Do something
}
```

6.2 Assign Booleans Implicitly

```
boolean shouldOrderFromZomato;

if(cashInWallet > 300) {
    shouldOrderFromZomato = true;
} else {
    shouldOrderFromZomato = false;
}
```

Instead of assigning variable to Boolean and then verifying whether it is True or False inside If-Loop.

We can directly check the value while naming the variable Boolean.

```
boolean shouldOrderFromZomato = cashInWallet > 300;
```

Remember while naming the variable as Boolean.

- Fewer Lines
- No Separate Initialization
- No repetition
- It should look like we are reading naturally.

Conditions – Don't be Anti-negative

While naming the conditions don't use negative words like

If (!notLoggedIn) {

}

Always use the positive conditions

If (loggedIn) {

}

6.3 Ternary is elegant

Always use DRY (Don't Repeat Yourself) approach while implementing the conditions.

For an example:

```
int deliveryFee;

if(isPrimeMember) {
    deliveryFee = 0;
} else {
    deliveryFee = 50;
}
```

Use this:

```
int deliveryFee = isPrimeMember ? 0 : 50;
```

Conditions – Don't be 'Stringly' typed, be Strongly typed

Always be strongly typed as Enums we can used anywhere in the condition without defining it.

Define Enums inside one class and then use it where needed.

```
public enum OrderStatus {
    INIT, IN_PROGRESS, IN_SHIPMENT, DELIVERED, PICKED_UP, RETURNED
}
```

```
if(orderStatus.equals(OrderStatus.DELIVERED)) {
    //Do something
}
```

And use it, instead of defining it, like this which is not proper way of doing.

```
if(orderStatus.equals("delivered")) {
    //Do something
}
```

When we use strongly typed, we can avoid

- No Typo's
- No Error prone
- Documents the states
- Searchable: Easy to find it

6.4 Magic Numbers

Magic numbers have the direct usage in the code.

For example, we have:

```
if(age > 18) {
    //Do something
}
```

It should be refactored to:

```
final int LEGAL_MARRIAGE_AGE = 18;

if(age > LEGAL_MARRIAGE_AGE) {
    //Do something
}
```

By using this, it improves readability of the code as well as easier to maintain. For an instance, let us consider the above example where we have set the value in legal marriage age field. If we have use magic number, so whenever the size changes, we have to change it at two different locations, so in-case if we forgot to change it at one then there will be consequences.

6.5 Complex Conditions

There are two ways you can simplify complex conditions as follows,

6.5.1 Intermediate Variable

When writing the conditions, we should not gently use intermediate variables directly, instead of that we should define it in the meaningful variable name and then use that variable in the conditions.

For an instance,

Here they have directly taken the numbers, which we should not do it.

```
if(employee.age > 55
        && employee.yearsEmployed > 20
        && employee.isRetired == true
) {
    //Do something
}
```

Instead of that we should define it in the variable and then use the variable in the conditions.

```
final int MIN_RETIREMENT_AGE = 55;
final int MIN_PENSION_EMPLOYMENT_YEARS = 20;

boolean isEligibleForPension = employee.age > MIN_RETIREMENT_AGE
        && employee.yearsEmployed > MIN_PENSION_EMPLOYMENT_YEARS
        && employee.isRetired;

if(isEligibleForPension) {
    //Do something
}
```

6.5.2 Encapsulate Complex Conditions

```
//Check for file extension. Also, check if Admin or File is Active
if((fileExtension.equals("mp4")
    || fileExtension.equals("mpg")
    || fileExtension.equals("avi"))
    && (isAdmin || isActiveFile)
) {
    //Do something
}
```

The above example has many conditional logics which have its logic in centralized, moreover which often results into bugs. By segregating the code into small parts will significantly improve the maintainability of the code. By refactoring it increases the comprehensibility, safety and support for the testing. The conditional logic should be encapsulated in the method and extracted.

Illustrating the above example,

```java
if(isValidFileRequest( fileExtension: "mp4",  isAdmin: true,  isActiveFile: false)) {
    //Do something
}

private static boolean isValidFileRequest(String fileExtension, boolean isAdmin, boolean isActiveFile) {
    boolean isValidFileExtension = fileExtension.equals(FileExtensionConstants.MP4)
            || fileExtension.equals(FileExtensionConstants.MPG)
            || fileExtension.equals(FileExtensionConstants.AVI);

    boolean hasPermission = isAdmin || isActiveFile;

    return isValidFileExtension && hasPermission;
}
```

Since, the method name describes the conditional, which improves the maintainability, hence we should refactor it.

6.6 Polymorphism over Enums

```java
public void loginUser(User user) {
    switch (user.userType) {
        case ADMIN:
            //Logic for Admin Login
            break;
        case CLIENT:
            //Logic for Client Login
            break;
        case RECRUITER:
            //Logic for Recruiter Login
            break;
        case CANDIDATE:
            //Logic for Candidate Login
            break;
    }
}
```

Instead of making ENUMS, we can directly use the polymorphism, through which we don't have to write down the specific logic for each one which is exactly the same logic applied to all the user.

Like in the above example, ADMIN, CLIENT, RECRUITER, CANDIDATE.

```java
public void loginUser(User user) {
    user.login();
}
```

We can make a method of user and then directly use it where this method can be extended by the specific user, as shown in the below example.

```java
class AdminUser extends User {
    public void login() {
        //Logic for Admin Login
    }
}

class ClientUser extends User {
    public void login() {
        //Logic for Client Login
    }
}

class RecruiterUser extends User {
    public void login() {
        //Logic for Recruiter Login
    }
}

class CandidateUser extends User {
    public void login() {
        //Logic for Candidate Login
    }
}
```

6.7 Be Declarative than iterative

In this approach, sometimes it is a good idea to go for declarative approach than iterative. However, it is not always possible with all the programming language that supports FP. So, if supported and if your code goes more readable, then go for this approach.

In the below example we are checking the conditions and filtering the user using the iterations.

```java
public List<User> getMatchingUsers(List<User> users) {
    List<User> matchingUsers = new ArrayList<>();

    for (User user: users) {
        if(user.accountBalance < MINIMUM_ACCOUNT_BALANCE
            && user.status.equals(Status.ACTIVE)
        ) {
            matchingUsers.add(user);
        }
    }

    return matchingUsers;
}
```

But instead of above we can write it as,

```java
public List<User> getUsersWithLowBalance(List<User> users) {
    return users.stream().filter(user->user.accountBalance < MINIMUM_ACCOUNT_BALANCE
            && user.status.equals(Status.ACTIVE)).collect (Collectors.toList());
}
```

Where we are filtering the data using the stream filter. Also, method name gives clear idea what the method does.

As using this kind of code, it increases the readability, and in the case where it is making too much of noise then use this approach.

6.8 Table Driven Methods

We generally write multiple if-conditions which is not needed, instead of that we can use the Table-Driven Approach.

```
if(currentAge == 50) {
    return 520;
} else if(currentAge == 40) {
    return 420;
}else if(currentAge == 30) {
    return 330;
}else if(currentAge == 20) {
    return 240;
}
```

In the below one, we use it where we can pass the Age to the repository function and based on that we will get rate which is defined.

insurance_rate_table

insurance_rate_id	age	rate
1	20	240
2	30	330
3	40	420
4	50	520

When should we use this approach?

- Great for full dynamic logic, where the rate is going to change based on the age.
- Avoids hard coding as we have written it in the databases, so if any changes appear, we can directly change it into the database.
- Write less code
- Easily changeable without code change or deployment.

This is only useful when we have dynamic logic as in the above example, we have rate.

7 Best Practices on Functions / Methods

When should we create a function?

- Duplication: when there is repetitive code.
- Excessive indentation: too much nested code, like if inside if
- Unclear Intent
- > 1 task: where we have task more than 1 which

7.1 Duplication

If your code contains the code duplication, there are ways to remove duplication of code as follows,

- Don't repeat yourself: when the usage of function is used in more methods, then create a separate function and call it inside the method
- Code is a liability
- Less is more

As we can see in the below example where the Date Time Formatter is used in each and every method, which decreases the intensity of the code.

```
DateTimeFormatter birthdateFormatter = DateTimeFormatter.ofPattern("MMM dd, yyyy  hh:mm a").
        withZone(ZoneId.of("Asia/Kolkata"));

String formattedBirthDate = birthdateFormatter.format(birthDate);


DateTimeFormatter joiningDateFormatter = DateTimeFormatter.ofPattern("MMM dd, yyyy  hh:mm a").
        withZone(ZoneId.of("Asia/Kolkata"));

String formattedJoiningDate = joiningDateFormatter.format(joiningDate);


DateTimeFormatter anniversaryDateFormatter = DateTimeFormatter.ofPattern("MMM dd, yyyy  hh:mm a").
        withZone(ZoneId.of("Asia/Kolkata"));

String formattedAnniversaryDate = anniversaryDateFormatter.format(anniversaryDate);
```

Instead of that we created a function and then we can use it where it is necessary.

```java
public String formatDate(Instant date, String timeZone) {

    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MMM dd, yyyy  hh:mm a").
        withZone(ZoneId.of(timeZone));

    return formatter.format(date);
}
```

7.2 Excessive Indentation

```
function iIds(startAt, showSessionRoot, iNewWnVal, endActionsVal, iStringVal, seqProp, htmlEncodeRegEx) {
    if (SbUtil.dateDisplayType === 'relative') {
        iRange();
    } else {
        iSelActionType();
    }
    iStringVal = notifyWindowTab;
    startAt = addSessionConfigs.siRange();
    showSessionRoot = addSessionConfigs.elHiddenVal();
    var headerDataPrevious = function(tabArray, iNm) {
        iPredicateVal.SBDB.deferCurrentSessionNotifyVal(function(evalOutMatchedTabUrlsVal) {
            if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                iPredicateVal.SBDB.normalizeTabList(function(appMsg) {
                    if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                        iPredicateVal.SBDB.detailTxt(function(evalOrientationVal) {
                            if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                iPredicateVal.SBDB.neutralizeWindowFocus(function(iToHenAddedCallback) {
                                    if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                        iPredicateVal.SBDB.evalSessionConfig2(function(sessionNm) {
                                            if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                iPredicateVal.SBDB.iWindow2TabIdx(function(iURLsStringVal) {
                                                    if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                        iPredicateVal.SBDB.idxTVal(undefined, iStringVal, function(getWindowIndex) {
                                                            if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                                addTabList(getWindowIndex.rows, iStringVal, showSessionRoot && showSessionRoot.length > 0 ? show
                                                                    if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                                        evalSAllowLogging(tabArray, iStringVal, showSessionRoot && showSessionRoot.length > 0 ?
                                                                            if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                                                BrowserAPI.getAllWindowsAndTabs(function(iSessionVal) {
                                                                                    if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                                                        SbUtil.currentSessionSrc(iSessionVal, undefined, function(initCurrentSe
                                                                                            if (!htmlEncodeRegEx || htmlEncodeRegEx == iContextTo) {
                                                                                                addSessionConfigs.renderMatchText(iSessionVal, iStringVal, eva
                                                                                                    id: -13,
                                                                                                    unfilteredWindowCount: initCurrentSessionCache,
                                                                                                    filteredWindowCount: iCtrl,
                                                                                                    unfilteredTabCount: parseTabConfig,
                                                                                                    filteredTabCount: evalRegisterValueSVal
                                                                                                }) : [], cacheSessionWindow, evalRateActionQualifier, undefined,
                                                                                                    if (seqProp) {
                                                                                                        seqProp();
                                                                                                    }
                                                                                                });
                                                                                            }
                                                                                        });
                                                                                    }
                                                                                });
                                                                            }
                                                                        });
                                                                    }
                                                                });
                                                            }
                                                        });
                                                    }
                                                });
                                            }
                                        }, showSessionRoot && showSessionRoot.length > 0 ? showSessionRoot : startAt ? [startAt] : []);
                                    }
                                });
                            }
                        });
                    }
                });
            }
        });
    }
```

When the code looks like an Arrow code, then we should definitely refactor the code as it decreases the complexity beyond 3 levels of nested ‘if’ as per a study in 1986.

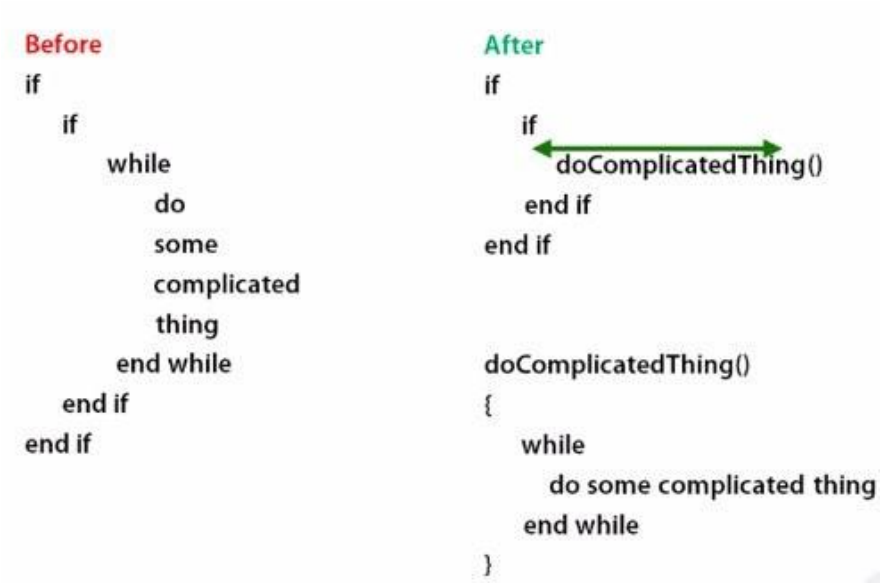
Always avoid the arrow code as it reduces the readability of the code and is also difficult to understand.

Solution of Excessive Indentation

- Extract method
- Return Early
- Fail Fast

7.2.1 Extract Method

If the code contains more than three levels of the indentation, then we should create a new method. For an example.



Extract method is easily readable and understand, which reduces the complexity.

7.2.2 Return Early

In a method if we write lines of code within multiple indentation after we return the single Boolean value that time, we have return early with the negative condition like

```
public boolean isValidUsername(String username) {
    boolean isValid = false;

    if(username.length() >= MIN_USERNAME_LENGTH) {
        if(username.length() <= MAX_USERNAME_LENGTH) {
            if(isAlphanumeric(username)) {
                if(!hasEasyLiterals(username)) {
                    if(isUnique(username)) {
                        isValid = true;
                    }
                }
            }
        }
    }

    return isValid;
}
```

```java
public boolean isValidUsername(String username) {

    if(username.length() < MIN_USERNAME_LENGTH) {
        return false;
    }

    if(username.length() > MAX_USERNAME_LENGTH) {
        return false;
    }

    if(!isAlphanumeric(username)) {
        return false;
    }

    if(hasEasyLiterals(username)) {
        return false;
    }

    if(!isUnique(username)) {
        return false;
    }

    return true;
}
```

7.2.3 Fail Fast

It is used for systems which should not fail as well as the application should not crash. Sometimes it fails, or crashes but try hard to prevent this.

- Delay the failure.
- Workaround when the failure happens so that software continues to work.
- Fail silently.

Why fail fast?

Fail-fast makes bugs and failures appear sooner, thus:

- Bugs are earlier to detect, easier to reproduce and faster to fix
- It's faster to stabilize software.
- Fewer bugs and defects will go into production, thus leading to higher-quality and more production-ready software.
- Fail silently

Bad example of the fail-fast is as shown below

```java
public void registerUser(String username, String password) throws BadRequestException {
    if(!isNullOrEmpty(username)) {
        if(!isNullOrEmpty(password)) {
            //Register user logic
        } else {
            throw new BadRequestException("Password is required!");
        }
    } else {
        throw new BadRequestException("Username is required!");
    }
}
```

In the above code the system can be fail as we have stated the negate the conditions is true.

A good example of fail-fast is as shown below.

```java
public void registerUser(String username, String password) throws BadRequestException {

    if(isNullOrEmpty(username)){
        throw new BadRequestException("Username is required!");
    }

    if(isNullOrEmpty(password)) {
        throw new BadRequestException("Password is required!");
    }

    //Register user logic
}
```

7.3 Convey Indent

```java
//Check for file extension. Also, check if Admin or File is Active
if((fileExtension.equals("mp4")
    || fileExtension.equals("mpg")
    || fileExtension.equals("avi"))
    && (isAdmin || isActiveFile)
) {
    //Do something
}
```

If Conditions has more than 2 parameters then we create another method for the validation.

```java
if(isValidFileRequest( fileExtension: "mp4",  isAdmin: true,  isActiveFile: false)) {
    //Do something
}
```

```java
private static boolean isValidFileRequest(String fileExtension, boolean isAdmin, boolean isActiveFile) {
    boolean isValidFileExtension = fileExtension.equals(FileExtensionConstants.MP4)
            || fileExtension.equals(FileExtensionConstants.MPG)
            || fileExtension.equals(FileExtensionConstants.AVI);

    boolean hasPermission = isAdmin || isActiveFile;

    return isValidFileExtension && hasPermission;
}
```

7.4 Do One Thing

```java
public String formatDate(Instant date, String timeZone) {

    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MMM dd, yyyy  hh:mm a").
        withZone(ZoneId.of(timeZone));

    return formatter.format(date);
}
```

Advantages of using it are:

- Eases reading
- Promotes reuse
- Avoids side effects

7.5 How many parameters function should have?

While defining the parameters in the function, the name of function should be what it will be functioning.

```
public void saveUser(User user, boolean sendEmail) {
    //Save user logic
    //Send email logic
}
```

Like in the below example save user should only save the username and same is applicable for the send email.

```
public void saveUser(User user) {
    //Save user logic
}

public void sendEmail(User user) {
    //Send email logic
}
```

Functions – too Long?

If function is too long that time function performs more than one task that can break the rule of clean code

long function name contains And, Or, If.

functions contain too many conditional branches

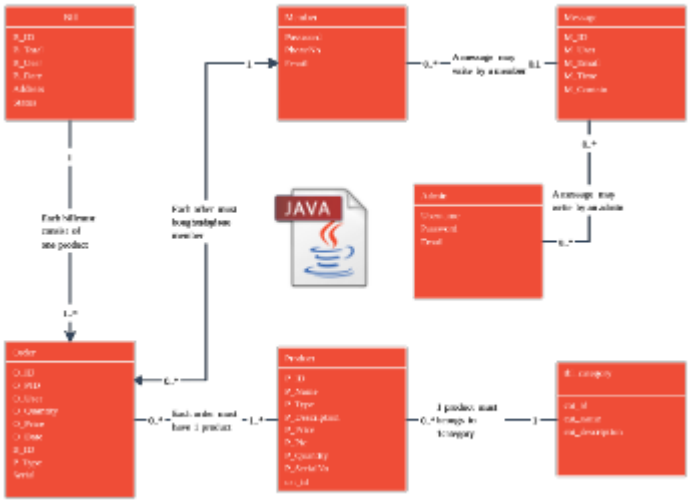
Too many whitespaces & comments.

Function is too long that time scrolling is required.

Guidelines of reduce function too long is

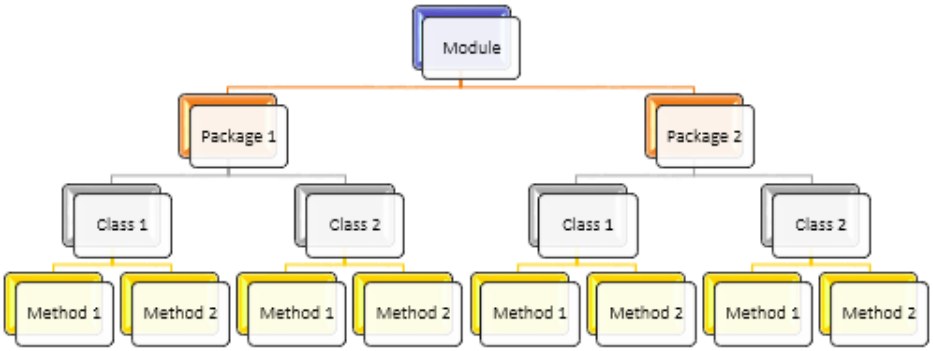
- Rarely be over 20 lines
- Hardly ever over 100 lines
- No more than 3 parameters.

8 Best Practices on Classes



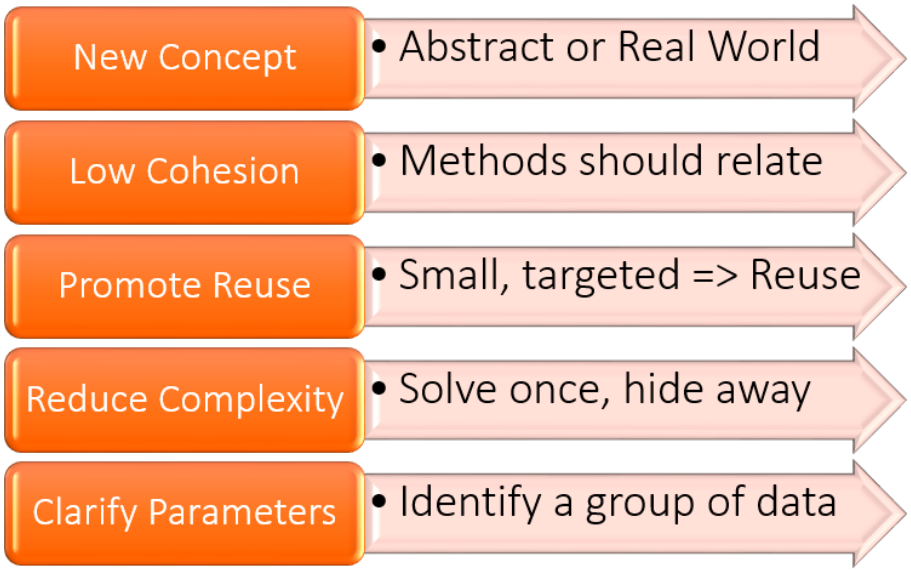
Classes are like heading in a book

Classes are like heading in the book as it gives us the brief knowledge of what it is going to contain, so the naming of classes should be appropriate where the name itself should tell what it is going to do.



8.1 When to create classes?

We should create a class when we want to organise code and data into logical modules, where logic is different, and it is applicable to other individually.



8.2 High Cohesion

High Cohesion refers to how closely all the methods or functions are closely related with a central purpose.

Class which has strong relationship between the modules or functionalities have High Cohesion.

Class responsibilities should be strongly related which

- Enhances the readability
- It increases the chances for reuse
- Avoids like lazy or clumsy.

While defining it we must watch out for these points.

- Methods which are standalone and doesn't have any relationship with the others.
- Fields which are only used by one method.
- Classes that change often.

8.3 Detecting low cohesive classes

Dirty class names

- Utility
- Common
- MyClass
- MyFunction
- HussainsObject

Low cohesive means when a class does a lot of jobs that don't have much in common

8.4 Primitive obsessions

Dirty code

```
public void saveUser(String firstName, String lastName, String email, String phoneNumber, String address) {
    //Save user logic
}
```

Clean code

```
public void saveUser(User user) {
    //Save user logic
}
```

Advantage of primitive obsession

- Helps to reader: reader easily understand the method or class uses.
- Implicit to Explicit class or object name.
- Encapsulation to easily get the value.

8.5 Principle of Proximity

The principle of proximity states that items close together are likely to be perceived as part of the same group

- Strive to make code read top to bottom when possible.
- Keep related actions together.

```
if(isValidFileRequest( fileExtension: "mp4",  isAdmin: true,  isActiveFile: false)) {
    //Do something
}

private static boolean isValidFileRequest(String fileExtension, boolean isAdmin, boolean isActiveFile) {
    boolean isValidFileExtension = fileExtension.equals(FileExtensionConstants.MP4)
            || fileExtension.equals(FileExtensionConstants.MPG)
            || fileExtension.equals(FileExtensionConstants.AVI);

    boolean hasPermission = isAdmin || isActiveFile;

    return isValidFileExtension && hasPermission;
}
```

9 Best Practices on Comments

Prefer Expressive code over comments.

Types of comments which should be avoided:

- Redundant
- Intent
- Apology
- Warning
- Zombie code
- Divider
- Brace tracker
- Defect log

9.1 Bad Comments

9.1.1 Redundant Comments

```
User adminUser = new AdminUser() ; //Instantiates new admin user
```

```
//<summary>
/* Default constructor */
//<summary>
public AdminUser() {
}
```

```
/* This will log in the user */
public void loginUser(User user) {
    user.login();
}
```

In Above code we assume that user can read and understand the code. Don't repeat yourself for comments

9.1.2 Intent Comments

In the below stated example we have assumed that user status is equal to one which means that user is activated.

```
//Assure user's account is activated.
if(user.status == 1)
```

But instead of that we can create a variable constant of active and then define it for the better understanding and readability.

```
if(user.status == Status.ACTIVE)
```

9.1.3 Apology Comments

Dirty

```
//Sorry, this code crashes a lot, so I'm just swallowing the exception

//I was too tired to refactor this spaghetti code
```

Don't apologize with comments, do not elaborate the issue.

Fix it before commit/merge

Add a TODO Marker if you must

```
//TODO: Move this configuration to properties file.
```

9.1.4 Warning Comments

Dirty

```
//WARNING: DO NOT CHANGE THIS IF YOU DO NOT KNOW WHAT YOU ARE DOING

//DON'T CHANGE THIS QUERY, THIS WILL RESULT IN ERROR.
```

To avoid warning, refactor complex code which has functionality that has indirectly support towards the system.

9.1.5 Zombie Code Comments

Dirty

```
    }
    List<KaptivateActivityCondition> conditionList = kaptivateActivityConditionRepository.findByActivityId(id);
    if (conditionList != null && !conditionList.isEmpty()) {
        conditionList.forEach(conditionObj -> {
            deleteActivityCondition(conditionObj.getId());
        });
    }
    kaptivateActivityRepository.deleteById(id);
    /* List<KaptivateActivity> listofKaptivateActivity = kaptivateActivityRepository.getAllActivityList(templateId);
    if (listofKaptivateActivity != null && !listofKaptivateActivity.isEmpty()) {
        int count = 0;
        for (KaptivateActivity activityObj : listofKaptivateActivity) {
            if (activityObj.getAnswerType().getDisplayName().equalsIgnoreCase("Welcome") || activityObj.getAnswerType().getDis
                kaptivateActivityRepository.save(activityObj);
            } else {
                count = count + 1;
                activityObj.setActivityOrder(count);
                kaptivateActivityRepository.save(activityObj);
            }
        }
    }*/
    return listOfMapActivity;
}
```

Common causes of zombie code

Risk aversion

Hoarding mentality

About comment out your code? Ask yourself?

- When, if ever, would this be uncommented?
- Can I just get it from GIT later on?
- Is this an uncomplete work to done in another branch?
- Is that configuration can be added in configuration file?
- Did I refactor out this code and it's outdated now?

9.1.6 Divider Comments

```
private void MyLongFunction()
{
    lots
    of
    code

    //Start search for available concert tickets

    lots
    of
    concert
    search
    code

    //End of concert ticket search

    lots
    more
    code
}
```

Do not write divider comments

It's better we create a new method through which user can understand it better.

9.1.7 Brace Tracker Comments

```
private void AuthenticateUsers()
{
    bool validLogin = false;
    //deeply
        //nested
            //code

            if (validLogin)
            {
                //Lots
                //of
                //code
                //to
                //log
                //user
                //in

            } //end user login

        //even
    //more code
}
```

Do not write brace tracker comments, try to create new method or better variable name which is easily understandable by the user.

9.1.8 Defect Log Comments

```
/* JIRA Ticket KTP-024: Implement Login Functionality, Date: 6th Feb, 2022 */
public void loginUser(User user) {
    user.login();
}
```

Do not write defect log, such as in the above example the whole JIRA ticket has been pasted which is not a good way to write a comment.

9.2 What are Good Comments?

```
//Encapsulates logic for calculating retiree benefits


//Generates custom newsletter emails
```

Good comments can be written for the better understanding of functionality.

10 Best Practices on Logs

Logging is very important part of the system development and separate topic to address. Here, in this section, some pointers are covered which are necessary while writing a code.

- Identify production troubleshooting scenarios in which logging will be helpful and based on that decide the right information and structure to be logged.
- Use proper logging levels: TRACE, DEBUG, INFO, WARN, ERROR
- Use standard logging library.
- Logs should be easily searchable:
    - Instead of long statements, prefer to write logs with short and concise words. “Key-value pair” based logging can be helpful.
    - Use the same wording for the same context. It is easy to read and analyze logs when they use the same words to express meaning.
- Do not include any confidential information in your log: passwords, tokens, or secret keys.
- Make logging part of your coding workflow. You will be thankful to have logs when your system goes down.

11 Staying Clean

You need to maintain your code to stay clean. Following are some of the ways to maintain your code,

11.1 When to refactor to Clean Code?

We should write clean code for the sake of the team rather than for future developers.

You're having difficulty understanding or changing.

You have enough time and knowledge to test the code.

11.2 Code Reviews

Promote proactive cleanliness.

Strive for readability.

Set Guidelines and Expectations.

11.3 Pair Programming

We should try to do programming in a pair,

- Which increases code quality
- It increases code quality
- It helps in refactoring the code.

11.4 Volunteer Rule

Always leave the code you’re editing a little better than you find it. - **Robert C. Martin**

12 Checklist

"No wise pilot, no matter how great his talent and experience, fails to use his checklist!"

For an engineer, following checklist has been created which he can use whenever he commits the code or creates a PR for review.

Based on the experience of previous projects, the most common pointers are kept in this checklist. When you adopt it for your project, kindly adjust them as per your needs.

Note: Because it is a checklist (and not guideline principles), it is important to check it every single time.

Sq No	Name	Category
1	I have used meaningful names of the variables/methods.	Naming Convention
2	I have enhanced the name of existing variables/methods based on better understanding of the requirements.	Code refactoring
3	I am not committing any unnecessary extra file/code/commented code.	Extra Code
4	I am not committing any confidential information.	Extra Code
5	I have written self-explanatory code and it doesn't require code comments to understand.	Code readability
6	I have added comments to describe business scenarios/business logic.	Code readability
7	I have followed "single responsibility principle".	Code readability
8	I have defined "folder by feature" structure, not "folder by type" structure.	Code readability
9	I have identified UnitTest Scenarios for the feature and at least documented them informally.	Functional Correctness
10	I have added meaningful logs for the new code. I have also added exception handling in the code.	Troubleshooting